Architecture-Centric Development

Copyright © 1998 Rational Software, all rights reserved

R

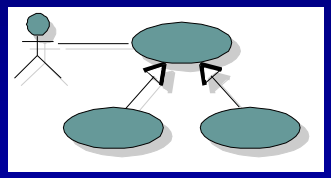
Objectives: Architecture-Centric Development

- Describe the “4+1 view” model of architecture
- Discuss the responsibilities of the architect
- List the contents of the Software Architecture Document
- Define patterns and frameworks
- Explain layered architectures and provide layering guidelines
- Describe how architectural mechanisms are identified and mapped to design and implementation mechanisms

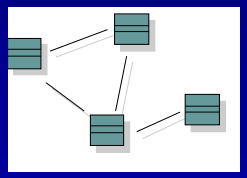
Copyright © 1998 Rational Software, all rights reserved

R

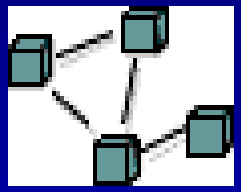
Architecture Is Represented by Views



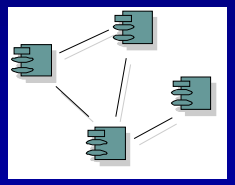
Use-Case Model



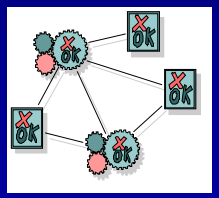
Design Model



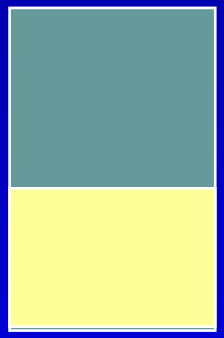
Deployment Model

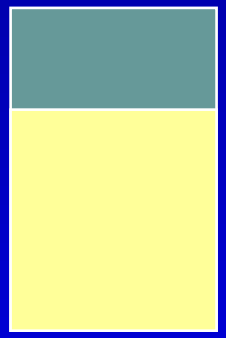


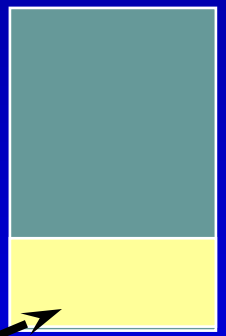
Implementation Model

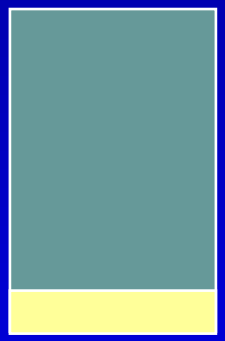


Test Model

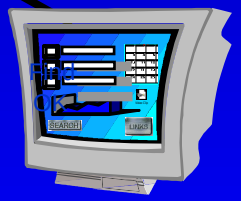


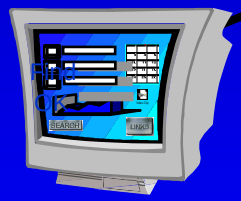






Use Case View

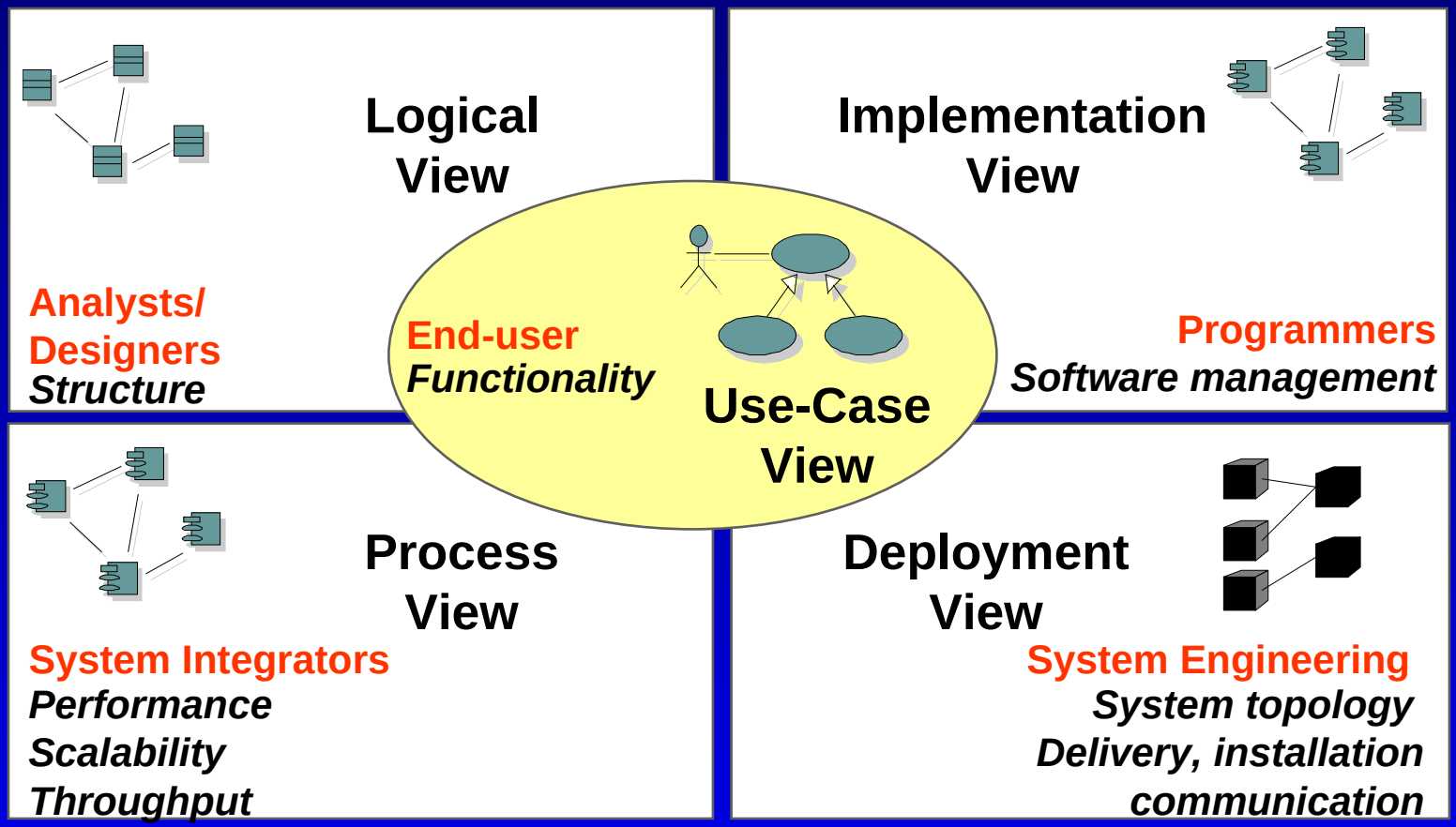




Implementation View

Copyright © 1998 Rational Software, all rights reserved

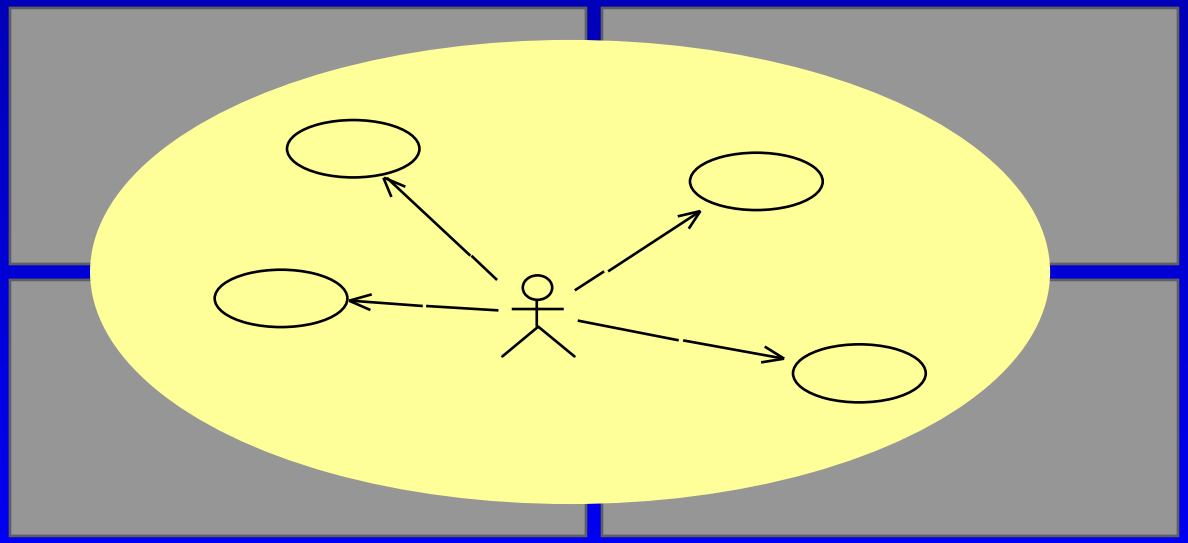
A “4+1 View” Model of Architecture



Copyright © 1998 Rational Software, all rights reserved

Use Case View

- Describes an architecturally significant subset of the use-case model (i.e. a subset of the use cases and packages)
- Concerns: functionality, critical functions, performance
- Represented, graphically, on Use Case Diagrams

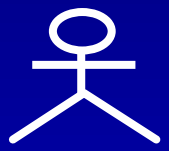


- **Actors**
- **Use Cases**

Copyright © 1998 Rational Software, all rights reserved

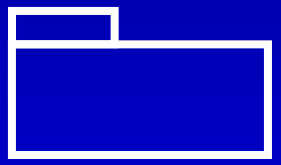
Use Case View Modeling Elements

Components



Actor

Use Case



Package

Connectors

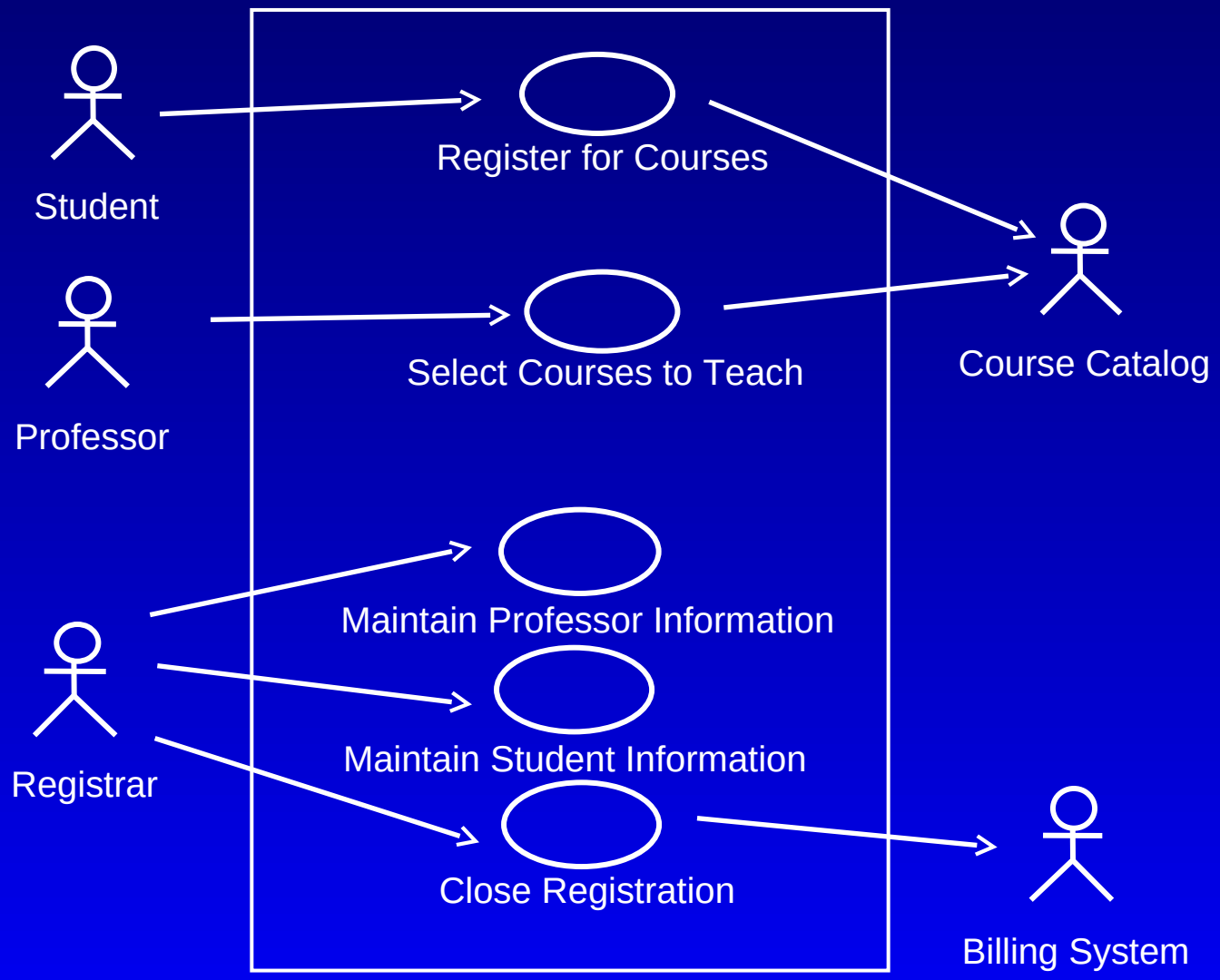
Generalization or Inheritance
<<uses>>
<<extends>>

Communicates

Copyright © 1998 Rational Software, all rights reserved

R

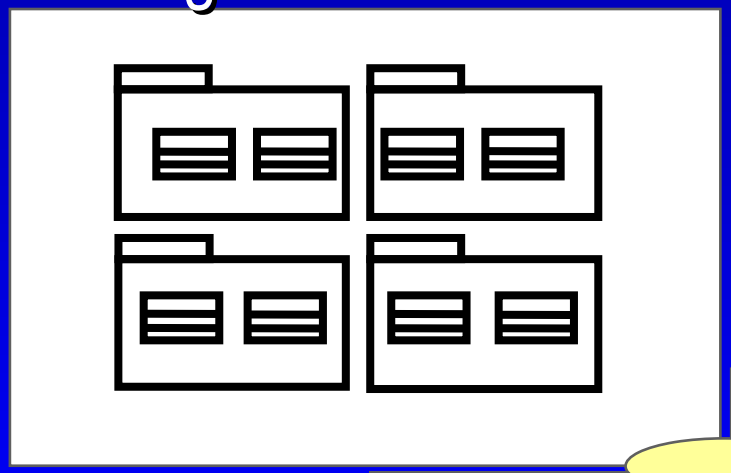
Use Case Diagram: Course Registration System



Copyright © 1998 Rational Software, all rights reserved

Logical View

- Describes an architecturally significant subset of the design model, i.e. a subset of the classes, packages, and use-case realizations.
- Concerns: functionality, behavior, use of frameworks, patterns
- Represented, graphically, on Class Diagrams, Interaction Diagrams and State Diagrams

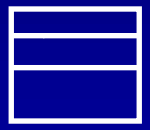


- **Package structure**
- **Interesting classes**
- **Use Case Realizations**

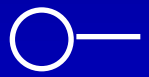
Copyright © 1998 Rational Software, all rights reserved

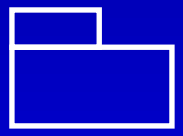
Logical View Modeling Elements

Components


class

state


interface


package

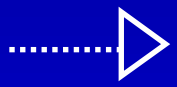
object

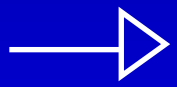
use case realization

Connectors

Association or link or transition

aggregation


realization


generalization

dependency

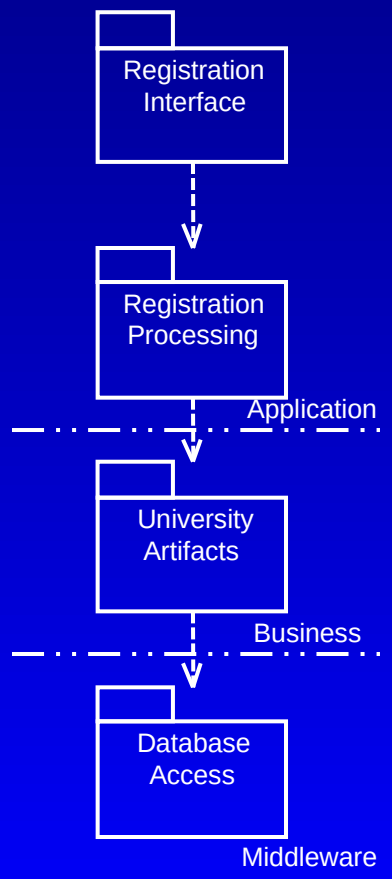
message

Copyright © 1998 Rational Software, all rights reserved

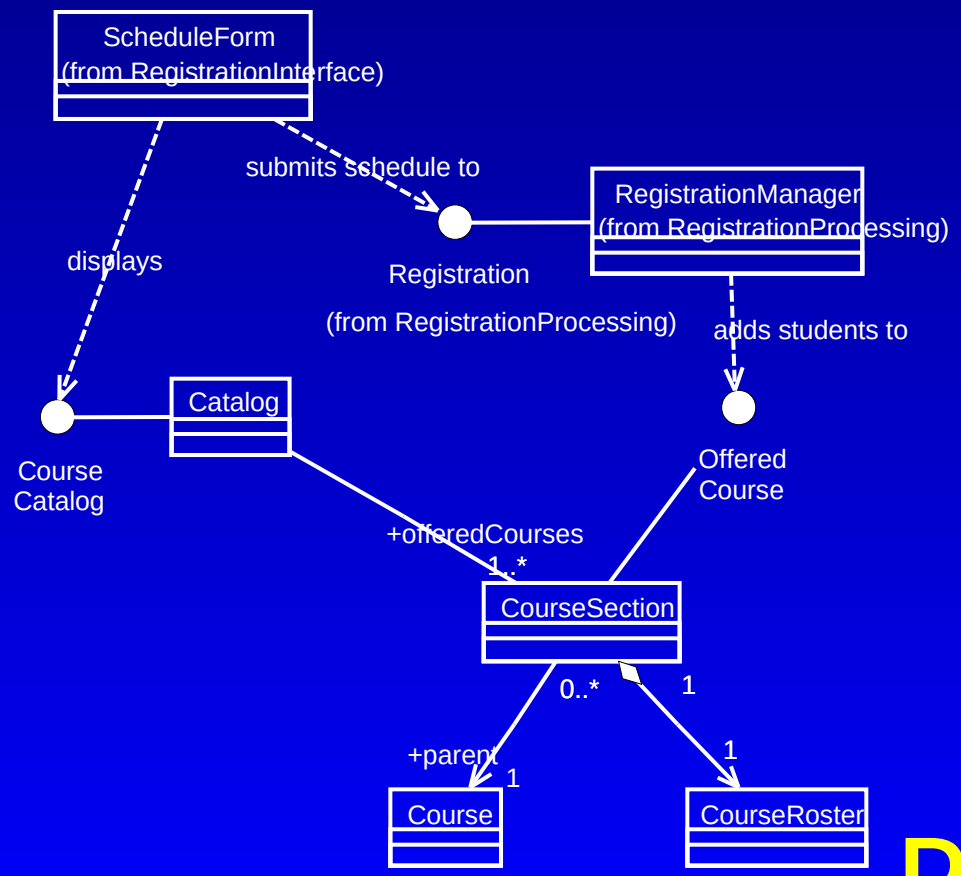
Class Diagram

- Registration System

Main Class Diagram



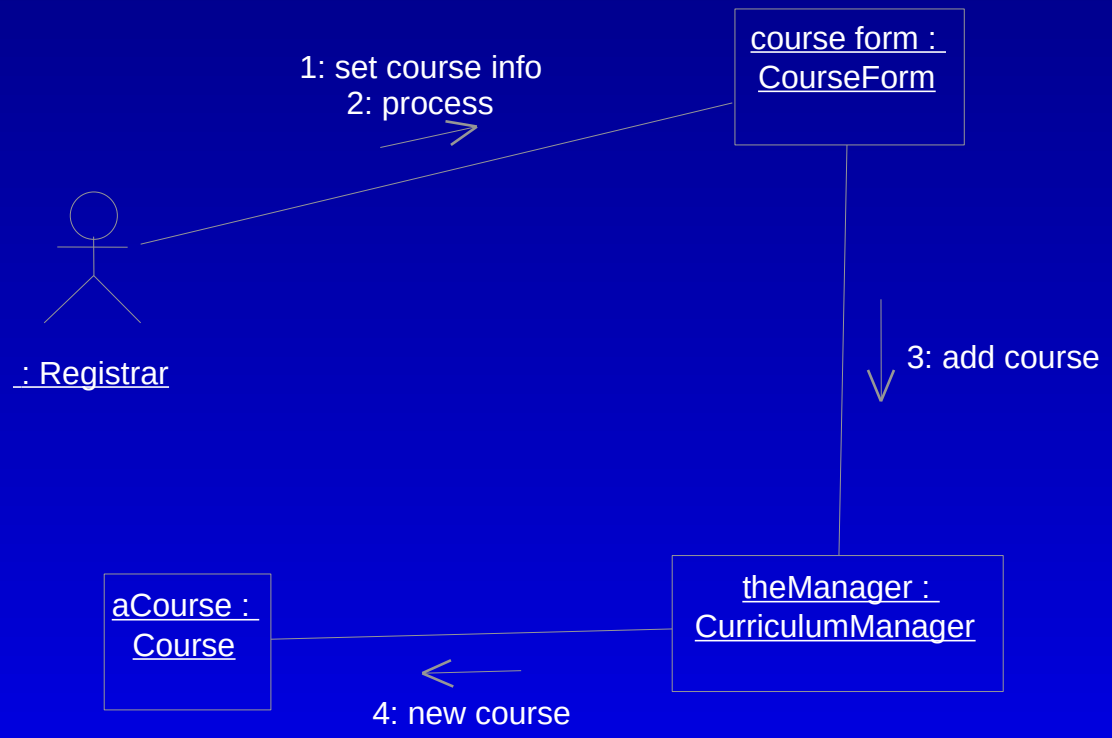
UniversityArtifacts Main Class Diagram



Copyright © 1998 Rational Software, all rights reserved

Collaboration Diagram

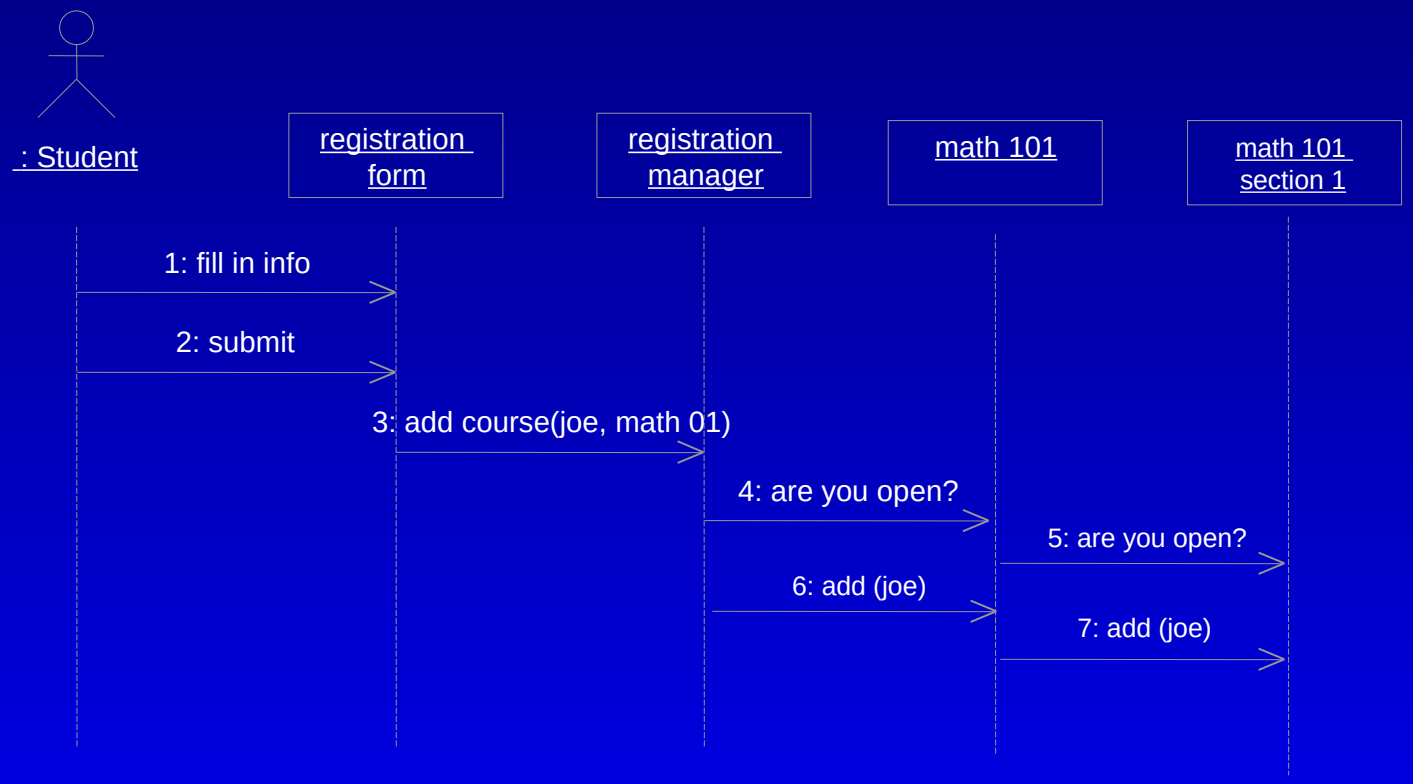
- Maintain Course Info Use Case (partial)



Copyright © 1998 Rational Software, all rights reserved

Sequence Diagram

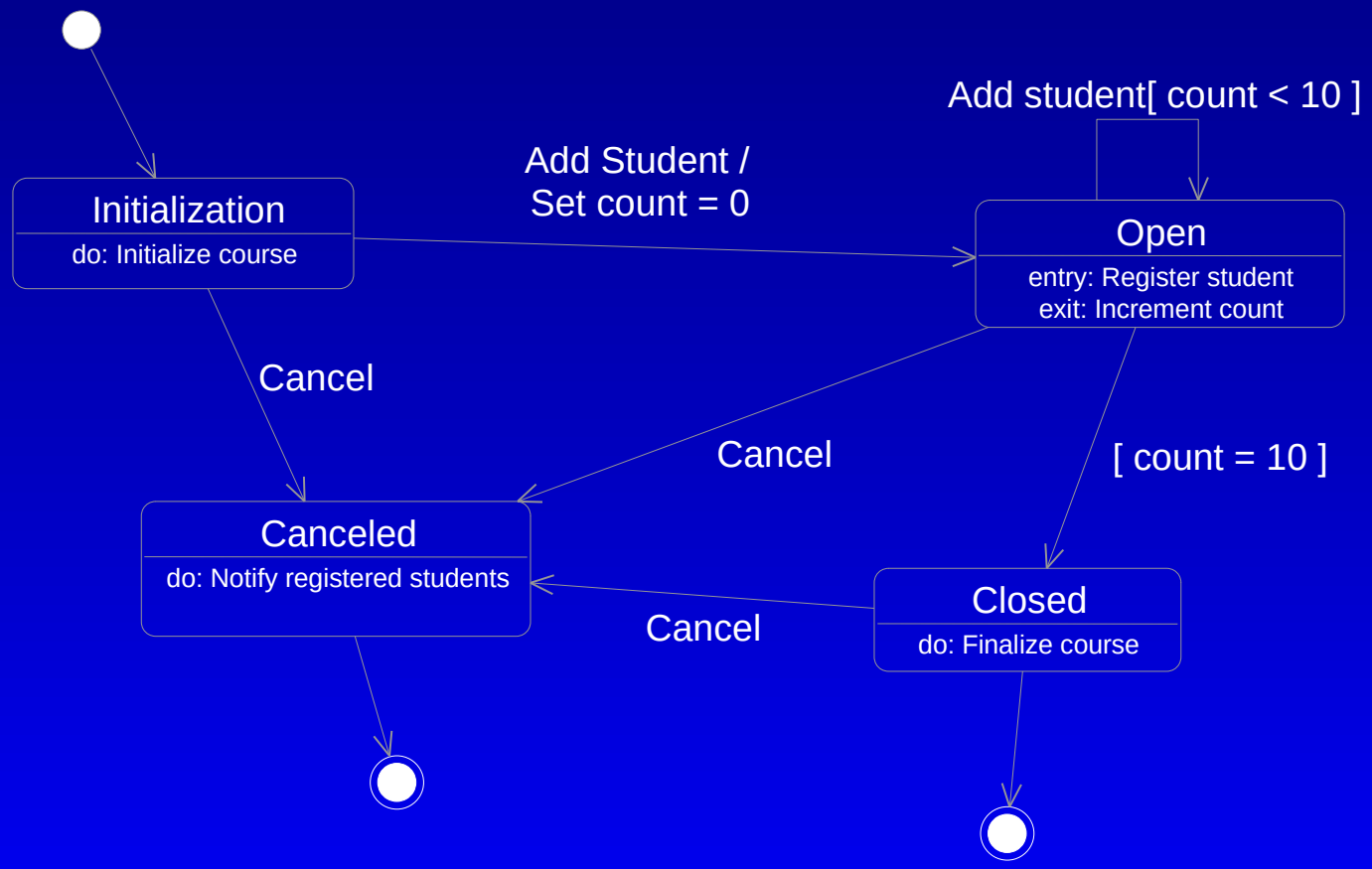
- Register for Courses Use Case (partial)



Copyright © 1998 Rational Software, all rights reserved

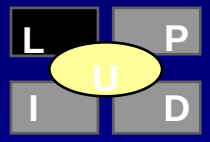
State Diagram

- State Diagram for the Course class



Copyright © 1998 Rational Software, all rights reserved

From Use Case View to Logical View



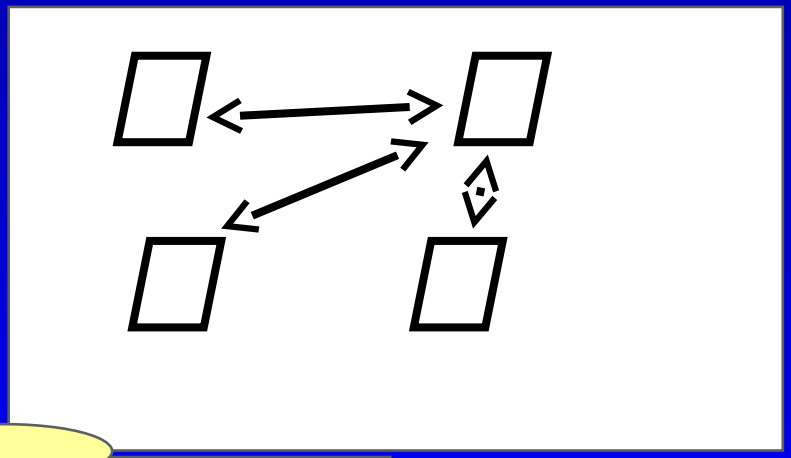
- Find Classes from Use Case Flow of Events
- Allocate the use case responsibilities amongst the identified classes
- Develop the use case realizations (e.g., “visually model” the use case)
- Identify the attributes and relationships needed to support the use cases
- “Package” the identified classes, refining package relationships, as necessary

Copyright © 1998 Rational Software, all rights reserved

Process View

- Describes the system's decomposition into executing tasks and processes
- Concerns: availability, reliability, scalability, integrity, performance, system management, synchronization
- Represented graphically on Component Diagrams and Class Diagrams

•Tasks
•Processes
•Process structure



Copyright © 1998 Rational Software, all rights reserved

Process View Modeling Elements

Component

Process1.exe

Process or Task (Active Object)

«stereotype»
Comp Name

<<process>> or <<task>>

interface

Connector

- - - - ->

Dependency(component diagram)

Link (collaboration diagram)

Message(Simple, Synch, Asynch, Timeout, Balking)

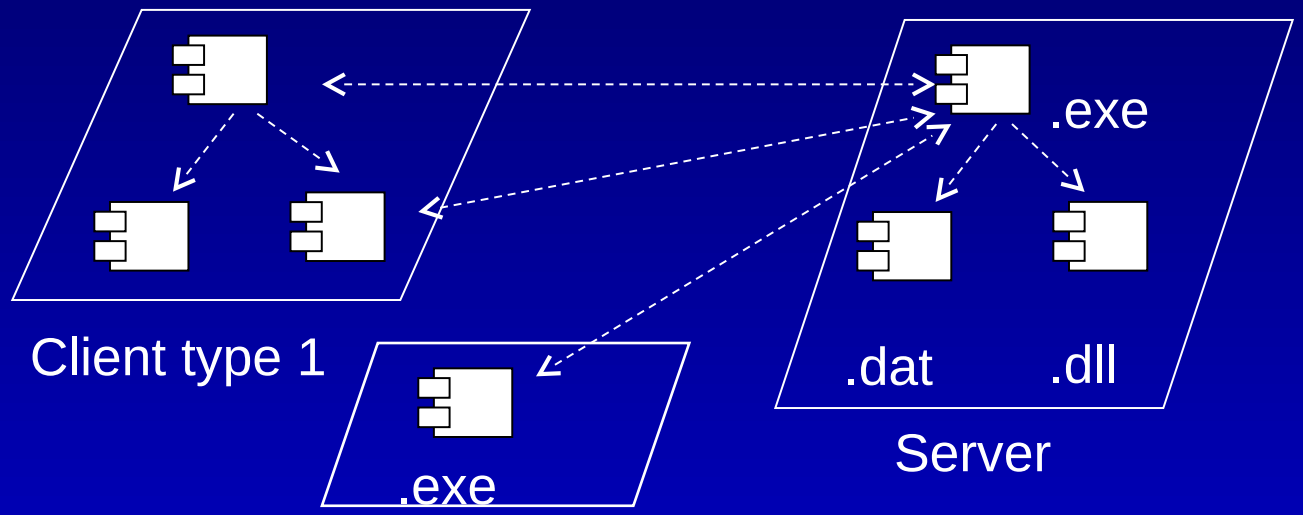
Copyright © 1998 Rational Software, all rights reserved

R

Process View Diagrams

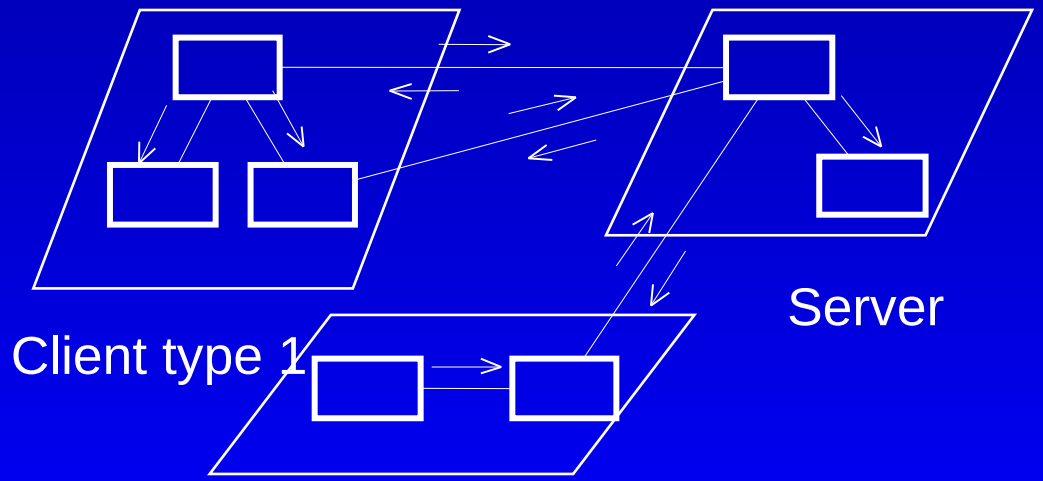
With Components

Component Diagram



With Active Objects

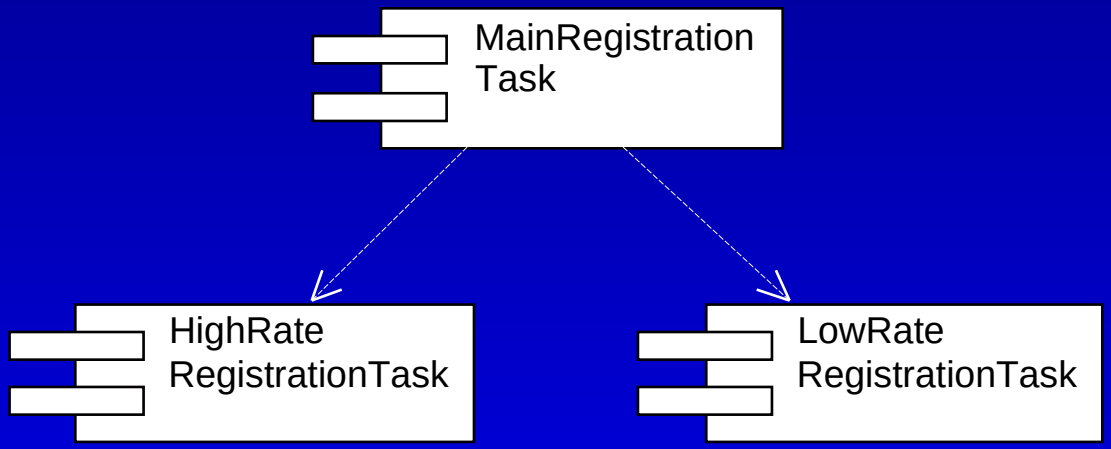
Collaboration Diagram



Copyright © 1998 Rational Software, all rights reserved

Component Diagram

- Registration Manager Process



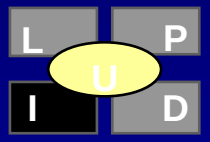
Copyright © 1998 Rational Software, all rights reserved

From Logical View to Process View

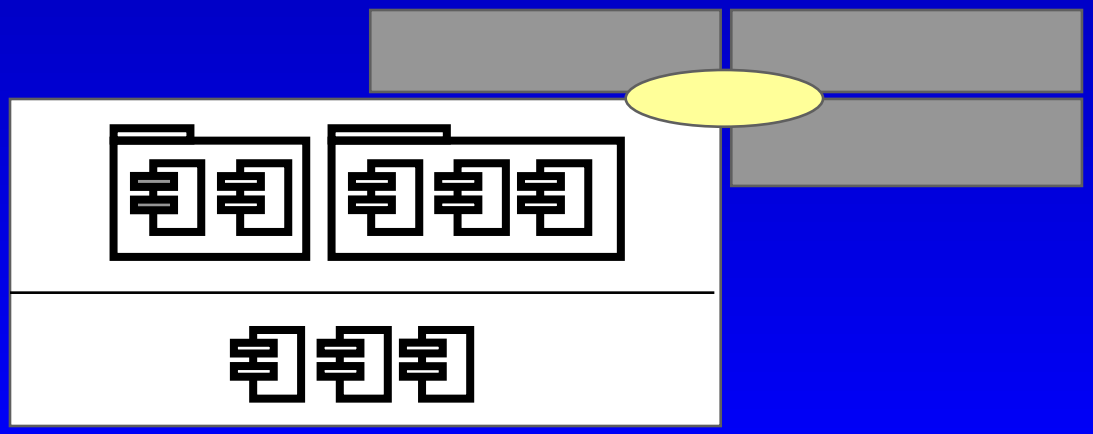
- A process for each active object is not possible
- Allocate active objects to processes based on:
  - Performance and concurrency requirements
  - Distribution requirements and support for parallel execution
  - Redundancy and availability requirements

Copyright © 1998 Rational Software, all rights reserved

Implementation View



- Describes the software component organization in the development environment
- Concerns: ease of development, team organization, inclusion of existing legacy systems or components, software configuration and management
- Represented graphically on Component Diagrams



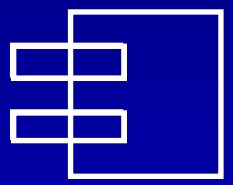
•Code library structure
•Source code

Deliverables
.exe
.DLL
Data files

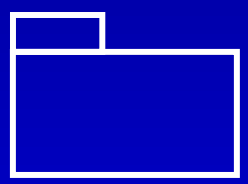
Copyright © 1998 Rational Software, all rights reserved

Notation for the Implementation View

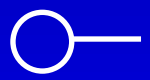
Components



Component
(such as .h, .cpp,
.dll, .exe,
.dat, etc.)



Package



interface

Connectors

.....>

Dependency
Compilation Dependency
(such as #include, “with”,
import, etc)

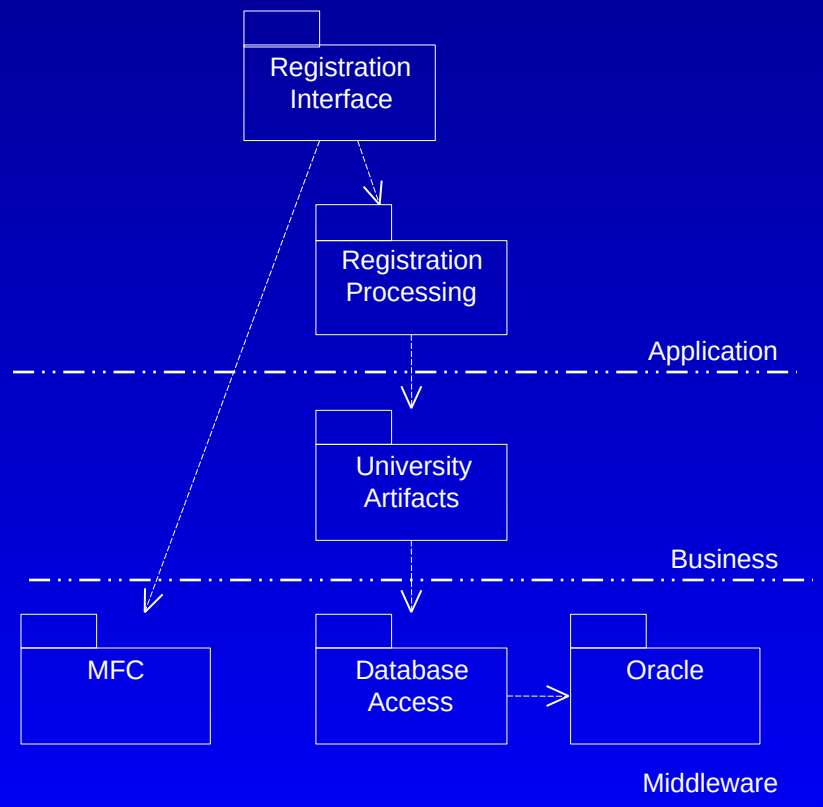
Copyright © 1998 Rational Software, all rights reserved

R

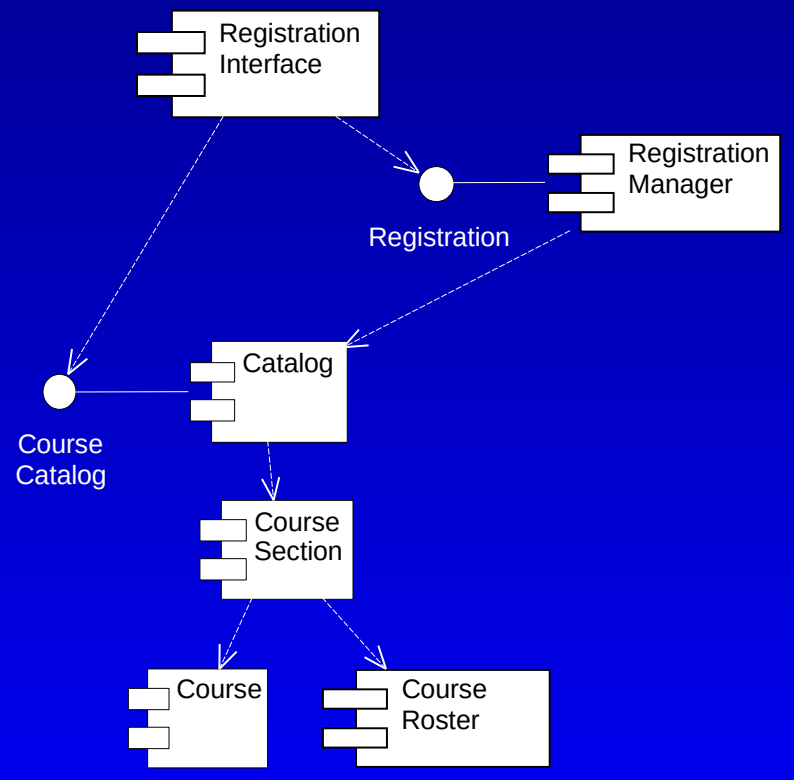
Component Diagram

- Registration System

Main Component Diagram

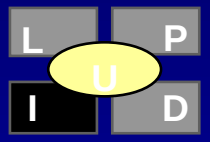


UniversityArtifacts Main Component Diagram



Copyright © 1998 Rational Software, all rights reserved

From Logical View to Implementation View

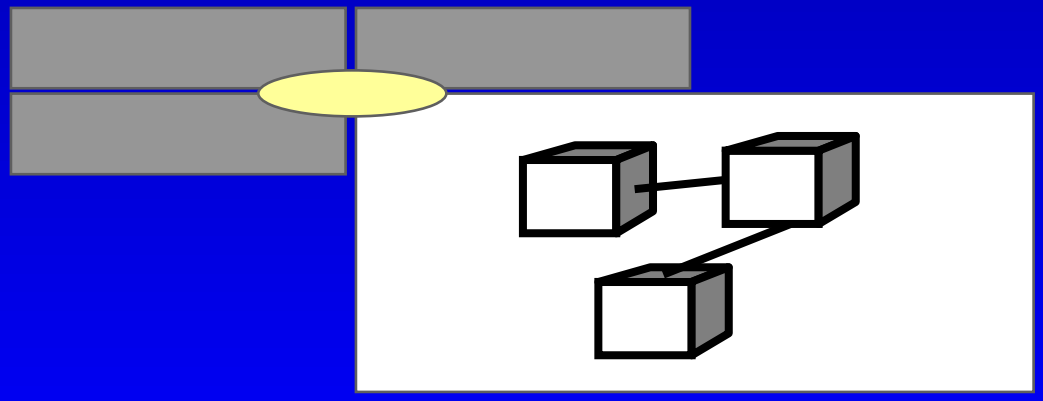


- Map Logical View elements to Implementation View elements
- Considerations:
  - Programming language
  - Team organization
  - Degree of expected reuse and commonality
  - Existing legacy systems or components
  - Strict layering principles
  - Release strategy/policy
  - Configuration Management

Copyright © 1998 Rational Software, all rights reserved

Deployment View

- Describes physical network configurations
  - Physical nodes and their interconnections
  - A mapping of the processes in the process view onto the physical nodes
- Concerns: performance, fault-tolerance, availability, installation, maintenance
- Represented graphically on Deployment Diagrams

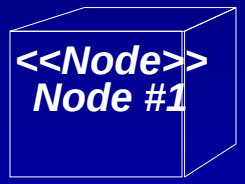


Hardware/ Software Mapping

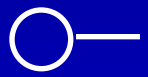
Copyright © 1998 Rational Software, all rights reserved

Deployment View Modeling Elements

Components



Node



interface

Runtime component

Process or Task
(Active Object)

Connectors

Communication association

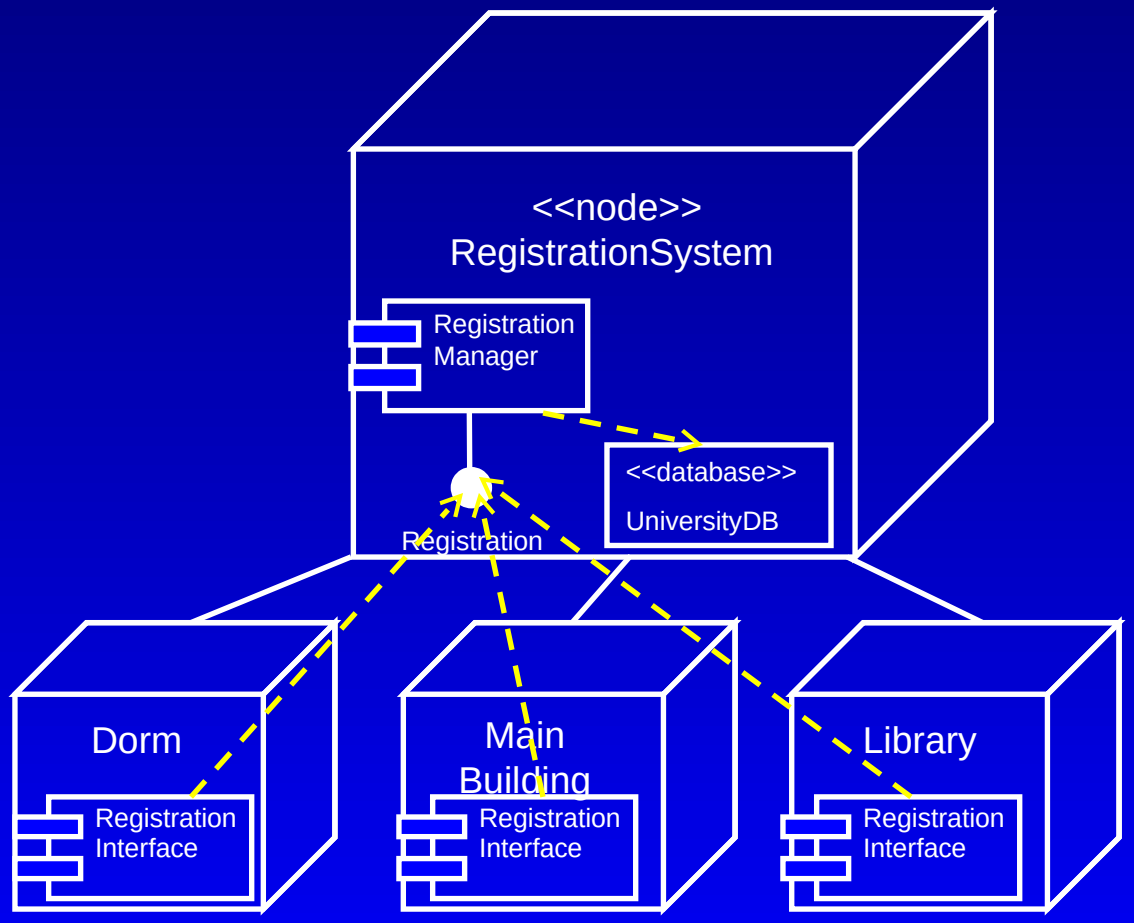
dependency

Copyright © 1998 Rational Software, all rights reserved

R

Deployment Diagram

- Registration System



Copyright © 1998 Rational Software, all rights reserved

From Process View to Deployment View

- Map the Process View processes, tasks and/or process groups onto the available physical hardware (nodes) for testing or deployment
- Considerations
  - Node capacity
  - Interconnection bandwidth
  - Availability (Primary/backup)
  - Rerouting
  - Load sharing
  - Parallelism

Copyright © 1998 Rational Software, all rights reserved

Key Worker: The Architect

- Leads and coordinates technical activities, including
  - Architectural analysis and design
  - Determination of system concurrency and distribution
  - Structuring of system models
  - Prioritizing use cases
- Is responsible for the overall content of the system models as well as:
  - Design Guidelines
  - Programming Guidelines
  - Software Architecture Document

Copyright © 1998 Rational Software, all rights reserved

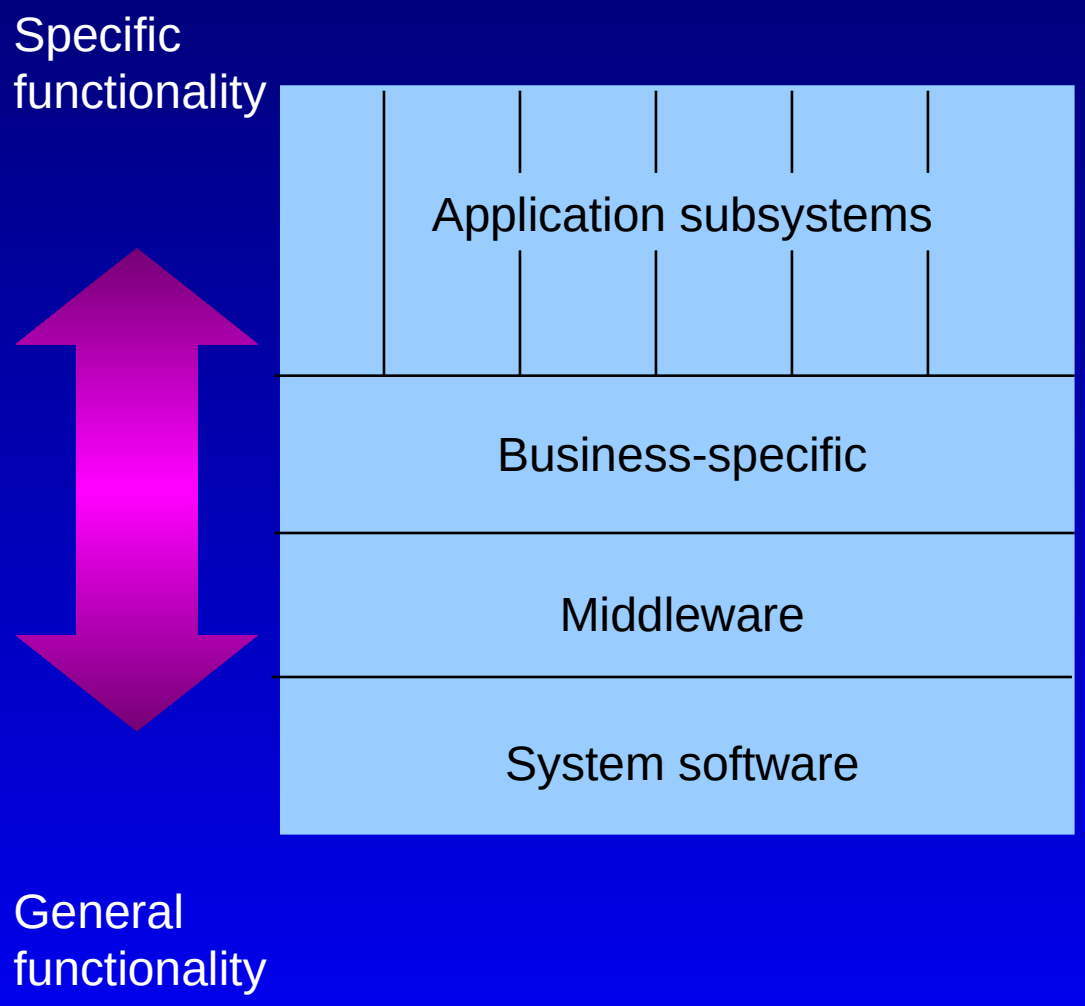
Key Artifact: The Software Architecture Document

- Comprehensive architectural overview of the system:
  - Architectural representation
  - Architectural goals and constraints
  - 4+1 architectural views
  - Data view (optional)
  - Size and performance
  - Quality

Copyright © 1998 Rational Software, all rights reserved

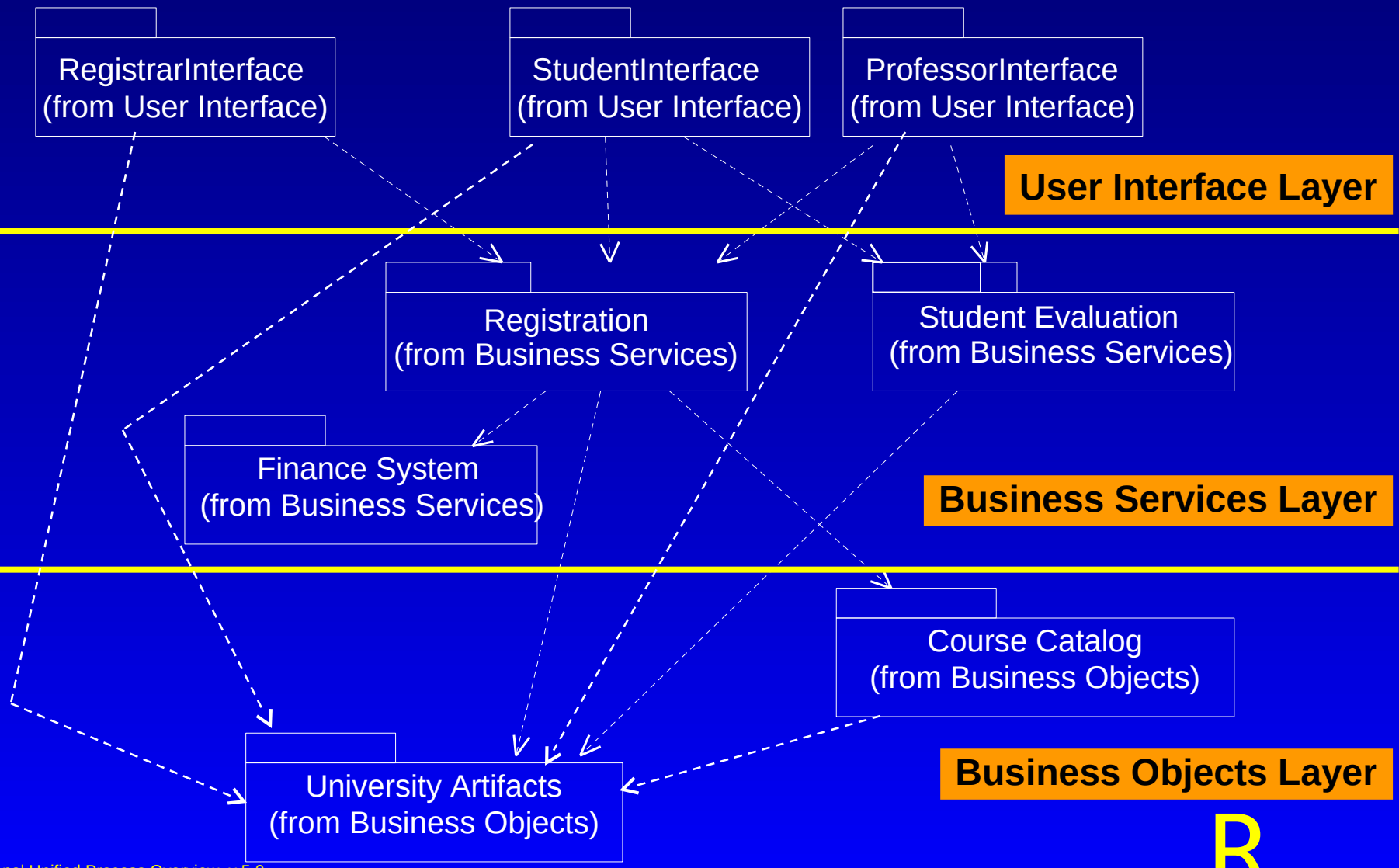
R

Layers: An Architectural Pattern



Copyright © 1998 Rational Software, all rights reserved

Example: A Layered Architecture



Copyright © 1998 Rational Software, all rights reserved

Layering Guidelines

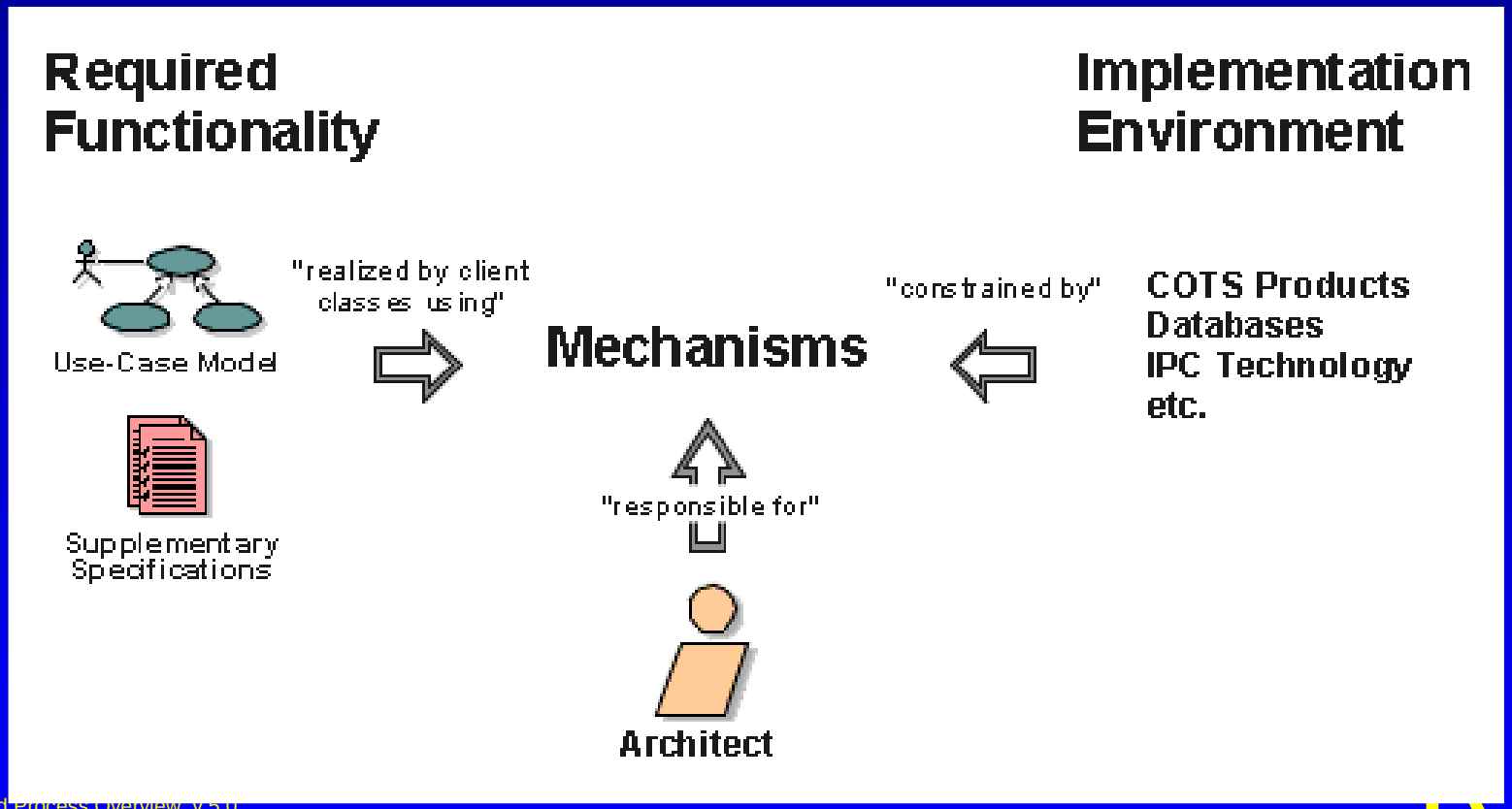
- Visibility
  - Dependencies only within current layer and directly below
- Volatility
  - Upper layers affected by requirements changes
  - Lower layers affected by environment changes
- Generality
  - More abstract model elements in lower layers
- Number of layers
  - Small system: 3 layers
  - Complex system: 5-7 layers

Goal is to reduce coupling and ease maintenance effort

Copyright © 1998 Rational Software, all rights reserved

Architectural Mechanisms

- Analysis Mechanisms (conceptual)
- Design Mechanisms (concrete)
- Implementation Mechanisms (actual)



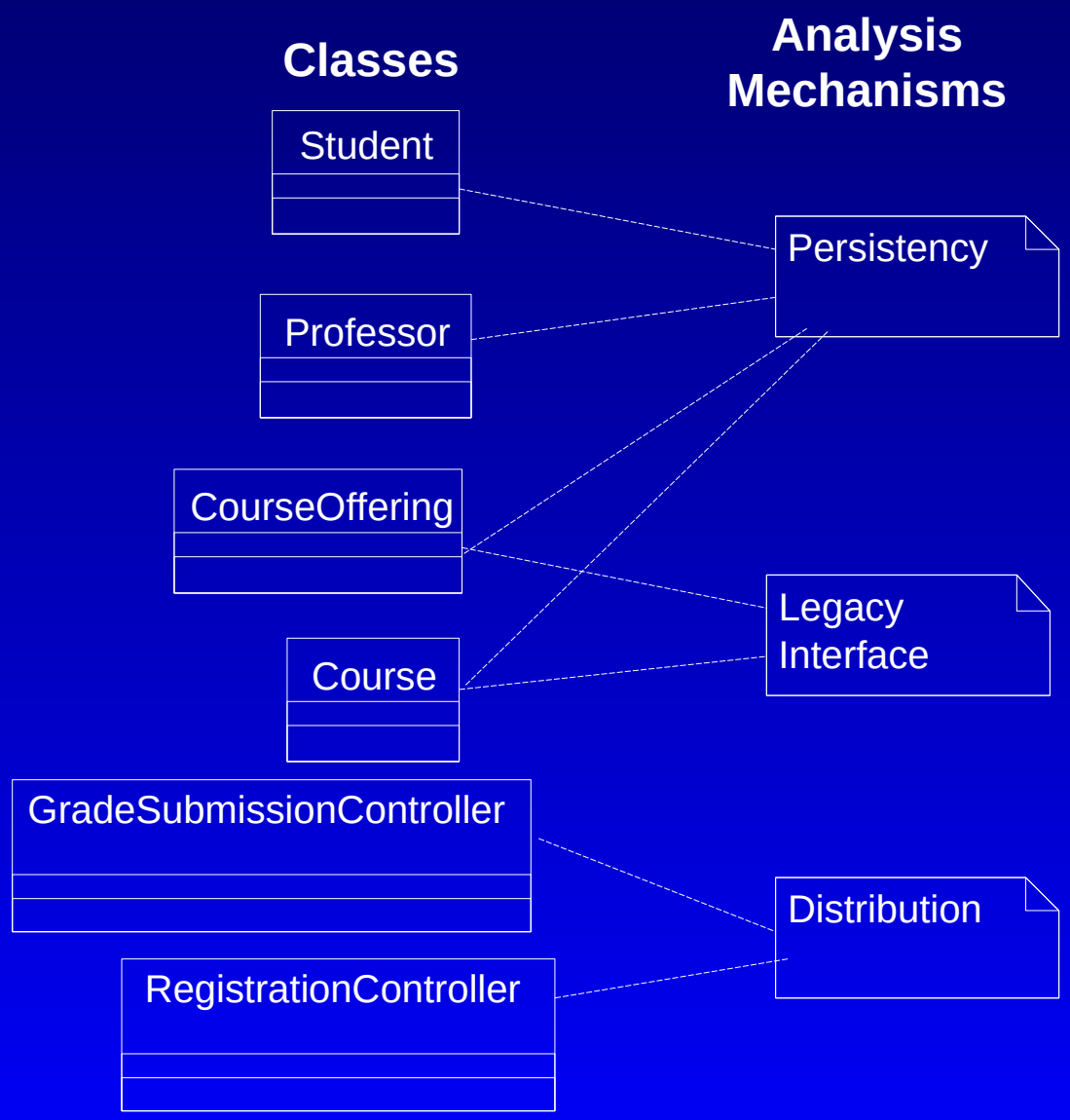
Copyright © 1998 Rational Software, all rights reserved

Sample Analysis Mechanisms

- Persistency
- Communication (IPC and RPC)
- Message routing
- Distribution
- Transaction management
- Process control and synchronization (resource contention)
- Information exchange, format conversion
- Security
- Error detection / handling / reporting
- Redundancy
- Legacy Interface

Copyright © 1998 Rational Software, all rights reserved

Mapping Classes to Analysis Mechanisms



Copyright © 1998 Rational Software, all rights reserved

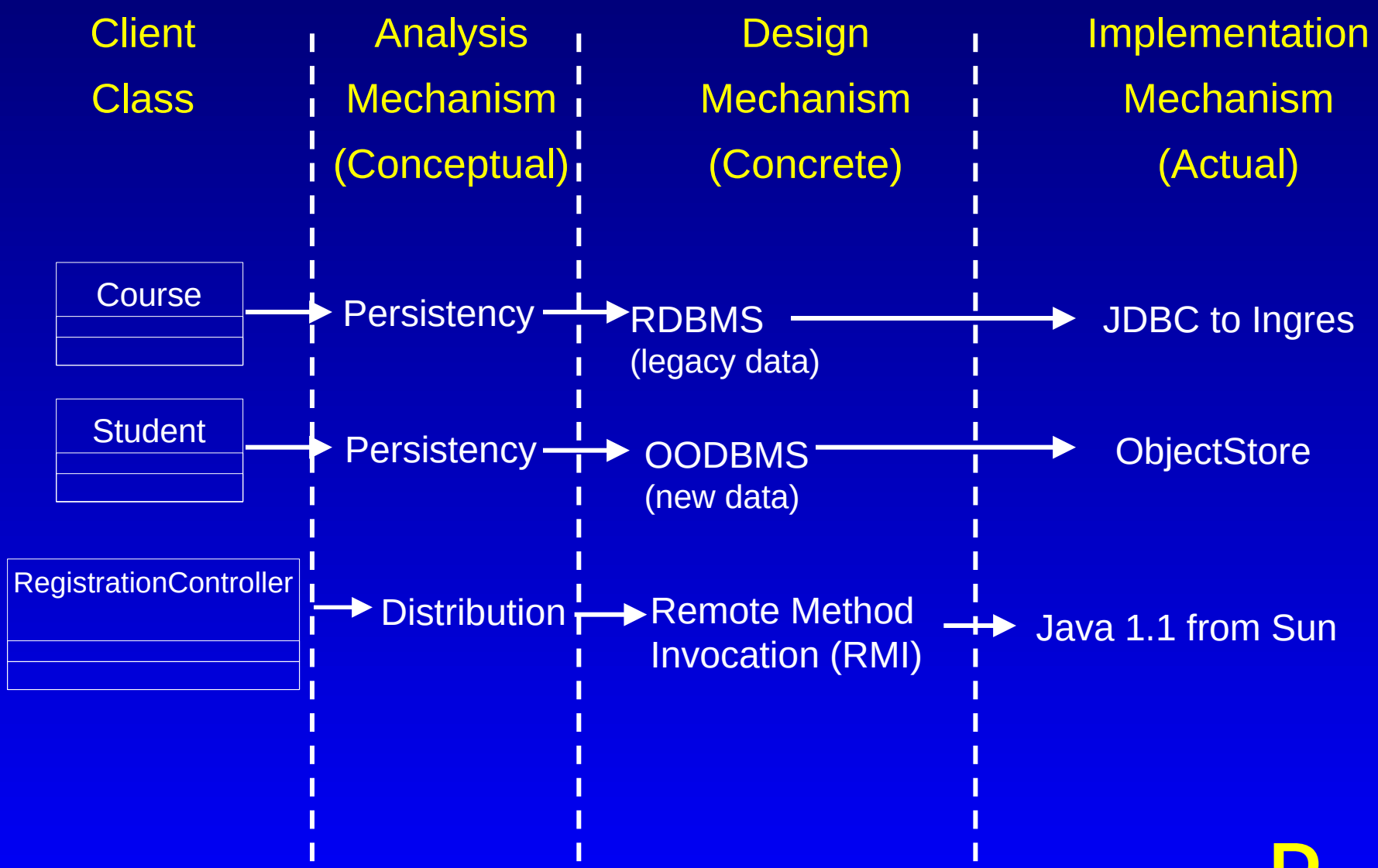
Sample Design Mechanisms

- For Persistency
  - In-Memory Storage
  - Flash Card
  - RDBMS
  - ODBMS
- For Communication and Distribution
  - Remote Method Invocation (RMI)
  - Shared memory
  - Function-call-like IPC , Semaphore-based IPC

Copyright © 1998 Rational Software, all rights reserved

R

End-to-End Architectural Mechanism Map



Copyright © 1998 Rational Software, all rights reserved

Frameworks and Patterns

- Framework
  - Definition of a general approach to solving the problem
  - Skeletal solution, whose details may be analysis/design patterns
- Pattern
  - A common solution to a common problem in a context
- Analysis/Design Pattern
  - A solution to a narrowly-scoped technical problem
  - A fragment of a solution, or a piece of the puzzle

Copyright © 1998 Rational Software, all rights reserved

R

Components Are Inseparable from Architecture

- Recall our definition of component:
  - A **component** is a non-trivial, nearly independent, and replaceable part of a system that fulfills a clear function in the context of a well-defined architecture
- Architectural mechanisms can be implemented as components and reused from application to application
- Business services and business objects are commonly implemented as components, intended for reuse
- Component-based development encompasses:
  - Building systems from reusable components
  - Developing and harvesting reusable components

Copyright © 1998 Rational Software, all rights reserved

An Architecture-Centric Process

- Architecture is the focus of the early iterations
- The architect is a key worker
- The Software Architecture Document is a key artifact
- Architectural activities span the requirements, analysis & design, implementation and project management workflows
- Architecture is the foundation for reuse
- Architecture governs design guidelines, including use of patterns and idioms
- Product structure derives from architectural groupings and interfaces
- Team structure also parallels architectural divisions

Copyright © 1998 Rational Software, all rights reserved

Summary: Architecture-Centric Development

- The “4+1” view model of architecture includes the logical, implementation, process, deployment, and use-case views
- The architect is a key contributor on the project, responsible for all aspects of architecture development and documentation
- Patterns and frameworks capture common solutions to common problems
- Layering is a way of organizing an architecture so that it is simple, understandable, and maintainable
- Architectural mechanisms provide common designs and implementations that can be used across a project

Copyright © 1998 Rational Software, all rights reserved

R